Chapter 19. SRE at the Tipping Point

It is now possible to instruct a machine to write code to do an arbitrary task using colloquial English. While not perfect, the output you get is generally usable, and we expect it will only get better.[1] This raises many questions.

Given that AI is redefining much of how we understand and approach software development, how is what we currently do in SRE likely to change as a result? As the underlying dynamics of development change, what is likely to happen to how software is built, maintained, and operated? How will nonfunctional requirements such as reliability and security be handled, and in particular, what will happen to production-related work? Or, to put it another way, if the cost of generating semi-useful lines of code becomes marginal, what becomes costly?

NOTE

Throughout the rest of this chapter, we will use the term "AI dev" to mean AI-driven software development (via CLI tool or editor plug-in, in-stream, or otherwise). This is distinct from "AI SRE," which we use to mean production engineering activities conducted with the aid of, and perhaps sometimes entirely by, AI.

The Software Development Lifecycle

Software, at scale, goes through a lifecycle as it is developed and deployed. While the precise details vary between organizations and projects, most software engineers are familiar with a prototypical model that divides software development into distinct phases, roughly along the following lines (as shown in Figure 19-1): *Plan* → *Analyze* → *Design* → *Build* → *Test* → *Deploy* → *Maintain*. This is a staged, or phased, model of the software development lifecycle; there are also SDLC models that are not linear, and of course, in practice, phases tend to overlap.



Figure 19-1. A simple linear software development lifecycle

NOTE

Although it wasn't present at the very birth of computing, the concept of a lifecycle model for software construction dates back about 60 years. A defining moment in its development was the famous 1968 NATO conference on software engineering, where the term "software crisis" was first used.[2] Simple but reasonable first-take models started to appear after that, probably the best known of them being Royce's waterfall model (1970).[3] A succession of other models emerged over the following decades, including the spiral model, Lean software development, continuous delivery, and many others.

A model discards some aspects of the world in order to represent a complex thing as a simpler thing that is easier to reason about, allowing us to make and test predictions about the world. Ultimately, the job of an SDLC model is to discard some details about how software gets done in order to give us ways to structure our work. It helps us partition and manage the complexity of a very complex task.

Two simple but powerful ideas to take from the SDLC are:

- Work is approximately *serialized*. There are certain stages that should be done before other stages, separating concerns—i.e., you shouldn't need to worry about testing while you're building, or about deployment while you're designing.
- There will be *loops*. That is, you might go back to certain earlier stages and cycle through them again. This is particularly relevant to the maintenance phase.

Together, these two ideas—though somewhat in tension—describe a good deal of the activity we do in software engineering: figuring out what to do and then how to do it, doing it, then going back to the beginning again.

The SDLC and Non-Functional Requirements

Performance, quality, reliability, security, testability, and so on are all important *non-functional requirements* of software, whose properties can be destroyed by changing a single line of code. As a result, those attributes cannot be ensured within the scope of just one phase; the work must take place across many phases.

Deciding how reliable something should be is a question of requirements gathering and resource optimization. As with functional requirements, these decisions are made during the Plan, Analyze, and Design stages. Ideally, SRE becomes involved during the Design phase (see Chapter 7), if not earlier.

Building a system capable of meeting those reliability targets requires work across the Design, Test, and Deploy phases—phases less central to pure functional concerns. SRE is often heavily involved in testing, and it is inherently and deeply engaged in deployment and maintenance.

The SDLC Under Tension and Transition

It appears that the cost of emitting moderately specification-compliant lines of code is being driven steadily closer to zero, and the speed of emitting those lines is also rising rapidly. What are the implications of these trends?

The SDLC phases can help us think about a world in which lines of code are produced much more quickly and cheaply (with, so far, mixed effects on quality). The phases themselves haven't changed: the same requirements still need to be met. But the time spent in each phase has shrunk, in some cases dramatically. As a result, the overall process creates much more output.

The increased speed and volume of software delivery are causing great excitement. However, we also discover a paradoxical effect: before AI dev, the work happening in each phase provided more time for incremental adjustment. A build phase lasting two weeks allowed you to make refinements while you were building, but a build phase of two hours doesn't.

Getting software in front of customers still helps answer many questions about what actually works, and now this can be done faster than ever before. But this helps primarily with the *functional* requirements, not the nonfunctional ones. The code being produced is currently about as reliable as it ever was—in fact, at the time of writing, there's some evidence that AI-generated code is less reliable—so the historical dynamics around defects ending up in production haven't changed. In other words, if we don't make additional changes to the way we work, reliability won't improve. Things will likely still break, and we'll still need to fix them.

Will this continue forever? We can assume that the software emitted by AI will remain imperfect—the code will not encompass all possible features, cover all possible failure scenarios, or implement all security requirements without error. Certainly, superhuman

AI coding performance may become available—but programs that execute successfully and produce the desired output aren't necessarily secure or reliable, and the training corpus probably contains many more examples of software that functions correctly than of software that both functions correctly and is highly reliable in production.

The Social SDLC

The SDLC is as much about social interaction and consensus building as it is about effective software delivery, describing communication paths between teams. This is explicit in the earliest stages—for example, the planning and analysis phases are mainly about understanding a business process, problem, or job to be done on behalf of a user, and almost always involve talking to someone. However, even the phases of the SDLC that involve a lot of mechanical support, such as testing, have a social component.

Conway's law states that communication between humans influences the structure of software. AI dev changes this. Communication no longer *has* to flow between teams or individuals to take software through the entire lifecycle: it is now possible for a single human, with the aid of a single agent, to take a specification all the way to production.

Although we try to establish an objective framework for definitions of concepts such as security and reliability, nonfunctional work also has a strong social component, relying on how people perceive them and the trade-offs they're willing to make. Many experienced SREs have sat in a post-incident review meeting discussing whether to prioritize paid over unpaid traffic, and there is no single objectively correct answer to such a question—it's often a matter of reaching consensus among the people in the room.

Adding AI to this situation *could* change it considerably. For example, in the scenario just described, AI could recommend a prioritization strategy that is itself subject to human approval; the entire prioritization process could be delegated to it (perhaps opaquely, perhaps not); or it could redesign the entire system so that prioritization never has to be done. Where social consensus previously shaped conventions that played out across the SDLC, AI-assisted software engineering will likely change this, including in production work. More of our work is based on conventions and rules of thumb than we often realize, and we should be prepared for AI-assisted software development to expose established ways of working to more rigorous scrutiny than they've ever received before.

SRE is particularly interested in the transition from testing to deployment. We expect the pre-deployment SDLC phases to become increasingly compressed as AI dev makes it easier to commingle activities that were previously separate. For example, it has long been considered good practice to run unit tests directly after writing new code, and many practitioners swore by test-driven development (TDD), while more comprehensive testing was left until later. However, running previously expensive, resource-hungry full-system tests essentially continuously is becoming close to best practice for managing AI systems. Automated evaluation is critical to many processes working at this speed, since humans no longer have the capacity to directly supervise them. In this world, building and testing converge.

Other SDLC phases are likely to converge as well. For example, knowledge gained during building can and should[4] influence how both humans and agents design software, rather than design influencing building unidirectionally. Agentic software objects as currently constructed are challenged by the need to strongly separate things, so it seems the direction is toward mixing them up. But the hard boundary imposed by deployment, and the consequences of getting it wrong, are unchanged, even if it's possible to produce lines of code at an accelerated rate. Organizations still need to deliver a quality user experience, are still exposed to different potential risks, may have certain legal or commercial obligations, want to make money from their software, and so on. The post-deployment world doesn't really care whether you are AI-ing or not.

So if the way we approach production remains unchanged, what does change? Here are our best guesses as to what is likely to happen in this nondeterministic future.

Rate of Change

Under AI dev, we will reach an important milestone: *the rate of code changes reaching production will very likely pass the point where human timescales allow for action.*

Tactically, this acceleration makes traditional cherry-picking increasingly difficult as a means of resolving production problems. This technique assumes that the development trunk is stable enough to select a subset of changes from which to build a custom binary. However, the trunk may be moving too fast to be considered stable, making this impractical. Rolling forward may become the primary method for changing production, with rollbacks less feasible, whether they are executed by humans or machines—the inverse of traditional SRE preference.

Critically, if *nothing else changes* but you speed up delivery by 10×, with high probability, you'll get 10× the incidents, 10× the weird interactions between subsystems, 10× the resource depletion, etc.[5] Expect to see more problems more rapidly —and if automation is developed to handle the easy cases, SREs will likely see the complex, difficult cases at approximately the rate they saw the easy ones in the past. (And how will you learn to handle the hard problems without having practiced on easy ones first?)

Of course, AI can help on the preventative side as well. Projects that proactively mine system topologies, source code, post-incident reports, and so on for ideas and practical approaches that can be applied across the board will likely become more common—perhaps in combination with approaches discussed elsewhere in this book, such as STPA. Similarly, we can leverage the stabilizing effects of AI by supplementing human review with test frameworks, eval frameworks, LLM cross-judging, and direct user feedback to help manage risk.

Strategic Incoherence and Technical Debt

Software behavior emerges from the interaction of services and systems, the data they consume, and user behavior. With more software, we'll likely see more of those emergent effects, and most testing approaches are poorly equipped to detect them. A key characteristic of today's AI dev output is a thing we call *strategic incoherence*: the systems are capable of performing quite sophisticated low-context tasks, but as their feature sets are extended again and again, their overall coherence tends to degrade. One common manifestation is that particular implementations get reproduced in slightly different ways throughout the codebase (violating the Don't Repeat Yourself principle), but this can occur on a subsystem level as well as a class level. There's also a potential negative feedback loop here: as duplication increases, system complexity may grow to overload the context window, and AI dev is unable to make meaningful progress under these conditions. In this way, AI dev systems can become tireless generators of technical debt—and technical debt often incurs an interest payment in production.

Approaches like specification-driven development are a way to overcome purely local optimization and incremental delivery. It's the same as in human-driven software engineering: documenting general design principles, system architecture, and so on helps both humans and AI systems stay aligned.

Versioning

Similarly, at *very* rapid change application rates, versioning as a way of partitioning risk starts to break down. By the time you've chosen a particular version of things (APIs, binaries, on-the-wire protocols), what's shipping already may have changed. There might have to be per-request, per-session, or per-user versioning. Answering questions like "What's actually running?" becomes nontrivial. Production resolution techniques that relied on stable versioning are less relevant. It is possible to live in such a universe, but it is very demanding.

One approach is revisiting how large-scale production environments were handled in the past. We went from one machine being up or down—a kind of Boolean model—to a production environment composed of hundreds or thousands of machines, some of which are up and some down at any given moment. We might treat the idea of versioning similarly: we don't need everything to be on one version, and in practice that's rarely the case anyway, but we do need to control the circumstances under which transitions occur.

Staging and Test Environments

Staging environments may also struggle to produce behavior usefully representative of production. If production scales substantially in size as well as in the rate and volume of changes (common as the total amount of deployed software increases), setting up and maintaining these environments could become infeasible.

Production Debugging

Production debugging is widely understood as the province of SRE. If the pace of change continues to accelerate as it has been, incident resolution and information gathering will likely require extensive tooling support. Furthermore, as the amount of deployed software grows exponentially, we can no longer rely on humans having read and reviewed enough of the codebase to build the context needed for production debugging. Instead, we will need to build that context on the fly, using tools (potentially lossy or probabilistic ones) to infer what happened. We will also need to infer the expected behavior on the fly, and most likely roll forward to the right behavior while the system continues to change underneath us.

If the rate of deployment continues to scale indefinitely, it will eventually become too problematic to do this manually at all. However, current tools are not designed to handle that scale either. This might ultimately motivate the construction of a large number

of separate production environments, which are automatically deleted and replaced as they fail when automatic recovery attempts are exhausted.

Socially and emotionally speaking, there is likely to be some negative impact on SREs, who derive part of their professional identity from being able to figure things out on their own in high-pressure situations. That said, tool-supported investigations may still inspire the same feelings.

Incidents

Until full artificial general intelligence (AGI) is available, we expect that humans will remain necessary to both prevent and respond to software failures. The primary reason is that software failures are inherently unpredictable, and current token-prediction AI techniques do not work well on out-of-sample cases. The unknown unknown category of incidents will continue to exist in any system where either the software or its inputs are changing, and that's doubly true when both are.

Current-generation LLMs have a nonzero probability of hallucinating, especially in complex situations, and public-facing outages often demand a certain degree of human accountability and explainability regardless of the capabilities of automated systems.

Incidents themselves are likely to take longer to resolve as well, since greater system complexity can result in longer and more complex causal chains. The old jokes about production outages being murder mysteries imply that we are heading more toward the world of multiple suspects being involved than lone wolves. Systems that track production changes are therefore likely to become even more important.

Another potential outcome is that incident response tools become so effective at resolving problems quickly that prevention efforts become underresourced. It's also possible that if responding to failures becomes cheap enough, organizations may start to place less value on systemic prevention, which is relatively more difficult.

Skill Erosion

Unexercised skills naturally lose fluency and relevance over time: we practice the Wheel of Misfortune (see Chapter 5) for a reason. New debugging tools and more complex scenarios won't change this.

Similarly, we may struggle with the difficulties inherent in relying on a machine whose suggestions are right only some proportion of the time. It's definitely easier to just press

that button—but sometimes as a result, things fail catastrophically. This risk is amplified because SREs tend to deal with rare and exceptional events at the tail end of the probability distribution. Tools that provide useful answers only for common or moderately uncommon events are not necessarily that useful for those of us who have to deal regularly with highly unusual events.

Observability and Monitoring

A bright spot is that the cost of introducing high-quality observability is likely to drop substantially. It's already easier than ever to introduce new metrics in code and follow that through to collection, aggregation, display, and so on. However, we are likely to see two obvious downsides from this: first, a proliferation of metrics, aggregations, and dashboards; and second, duplication of those metrics, as AI dev systems are unlikely to consistently deduplicate metric generation at the source. Conversely, AI SRE is very likely to become the de facto observability layer on top of a set of ever-changing—possibly even dynamically generated—telemetry.

The observability of AI systems themselves remains a hugely complex problem. Although it may well get better, there is still considerable work to do before we can reliably understand what agentic systems did, how they did it, and why. The *why* is, unfortunately, critical to establish for production changes, and without that context, the search space for debugging increases dramatically.

Formal Methods

Another bright spot is that using formal methods to describe system behavior is becoming increasingly tractable and efficient. Systems that are constructed in this way can provide guarantees about certain aspects of their behavior, making them more reliable than systems that are not (although formal methods will still need to be complemented by comprehensive testing). We regard this as a very promising area and talk about it more in Chapter 14.

Leadership and the Interpretation of Value

This harks back to questions of the social construction of value. To be clear, we see nothing that indicates to us that the job of running production is going to become less important or less difficult. But we do anticipate that SREs are going to be equipped with many more tools to do their jobs. It might well become impossible to do the job without

those tools. Furthermore, while innovation is necessary, it seems unlikely that all of those tools can be successfully agentic, at least in the short term.

However, we are practitioners, and we naturally have a strong bias toward valuing that activity. It is possible that as the work mix changes, and agentic SRE takes on more of the post-deployment activity, what we do may also change. For example, we have previously likened the role of the Google SRE to what the outside world calls a "staff engineer": someone who is skilled with code but maintains a holistic view of the software and the teams that build and operate it. Perhaps the pre-deployment work of SREs will become even more important.

Conclusion

The way we make software is likely to continue to change, and the constraints that motivate how we do so are becoming less and less relevant. As a result, nonfunctional requirements in general, and specifically how SRE ends up fulfilling its traditional mission, are also likely to change.

In the absence of AGI and perfect pull requests, the constraints of the production domain are likely here to stay. It is meaningful to push something to production. Every time you do so is an opportunity for the world model envisioned by the change to fail, and at the time of writing, AI dev does not have a great world model itself. AI SRE tooling may help with some of that, but there will be more (and more complex) unknown unknowns as the size of the complexity space grows exponentially along with the size of the codebase. Incidents are likely to be the front lines of this conflict, and it seems likely that even excellent-quality agents will be unable to completely avoid creating difficult-to-resolve problems—even our best teams struggle with that today. At the moment, humans as the understanders-of-last-resort is the best approach we have, even if we move from comprehension at the moment of creation to comprehension at the moment of failure.

Just as the SDLC is now ripe for reworking, SRE itself is at a tipping point. The assumptions underlying much of our work are being invalidated, and the way forward is not yet fully clear. But we believe that certain things are not going to change—the constraints of production, in particular—and in a sense, AI SRE is assistance for what has always been very demanding work. If as a result we all become more like staff engineers—i.e., individuals with a holistic approach to a system, people who know it both broadly and deeply—and are still professionals steeped in software, we might even welcome that change.

[1] Questions remain about how quickly the quality will improve, which domains it works well and poorly for, and the larger social and environmental effects that are likely to flow from all this; this chapter will not tackle them.

[2] See section 7.1.2 in the conference notes.

[3] Ironically, given its subsequent widespread adoption, Royce explicitly described the waterfall model as "risky and invit[ing] failure," proposing several other features to mitigate the risks.

[4] Royce argued this in the original waterfall model paper, for example.

[5] The relationship might not be fully linear, but it seems unlikely that we'll get lucky with it being sublinear.

Previous chapter
< VI. Outlook

Next chapter
20. How SRE Supports AI >